

NHR SÜD-WEST

INTRODUCTION TO PARALLEL I/O AND DISTRIBUTED FILE SYSTEMS

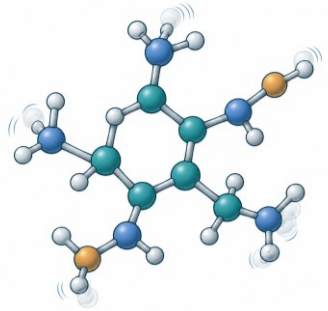
Sarah Neuwirth
Johannes Gutenberg University Mainz

November 2, 2026

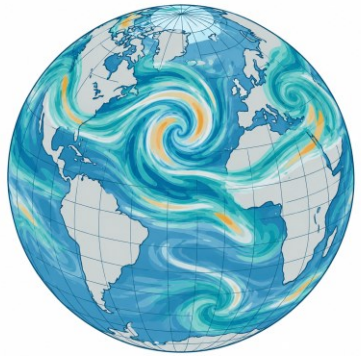
JG|U

MOTIVATION
Why is Scientific I/O so Difficult?

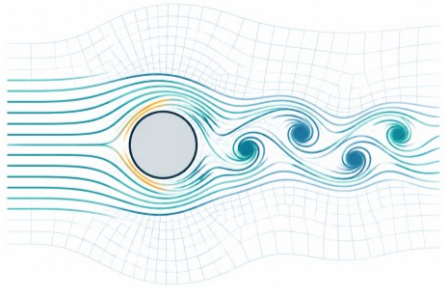
- Scientists think in terms of **molecules, particles, grid cells and physical fields.**
- Applications represent these as **arrays, meshes and other data structures**, often distributed across many processes.
- The I/O stack translates this structure into **file offsets and byte transfers**.
- Performance depends on matching **data layout, request sizes and concurrent access** to the storage system.



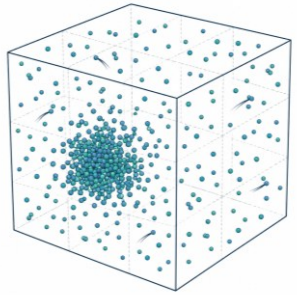
Molecular dynamics.



Weather and climate modeling.



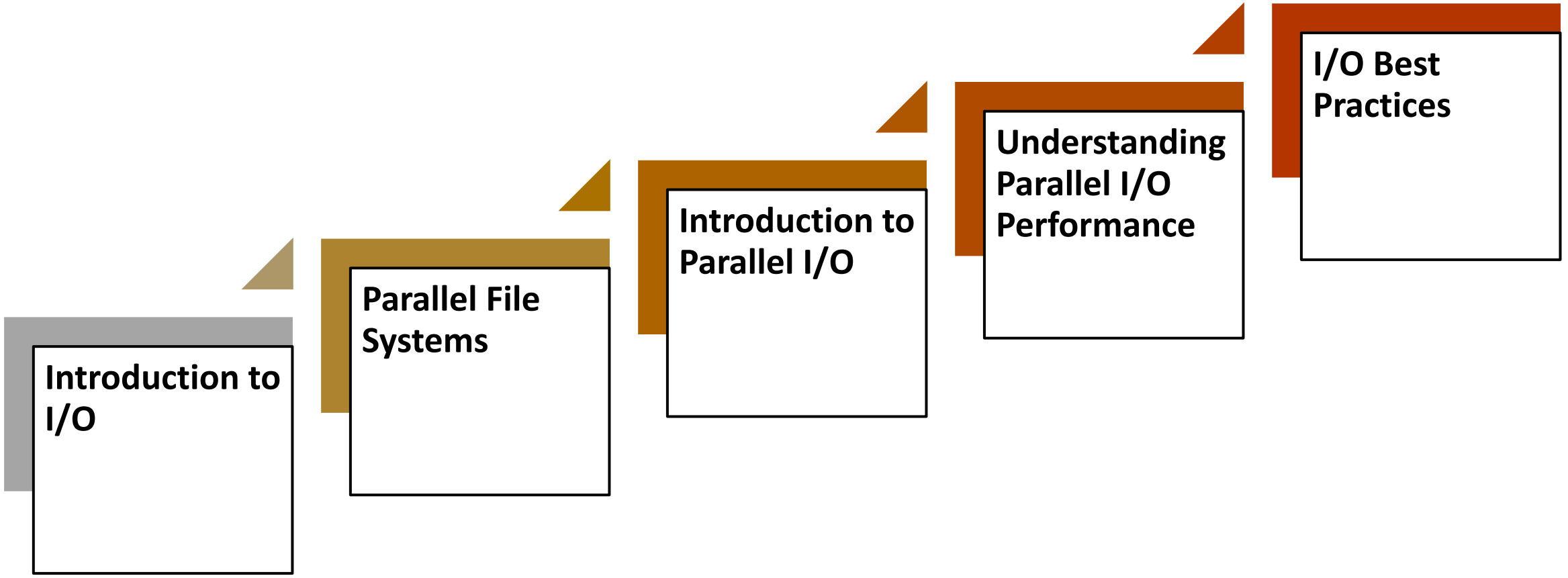
Computational fluid dynamics.



Particle simulations.

Concept adapted from Rob Latham et al., "Parallel I/O in Practice", SC Tutorial.

OUTLINE

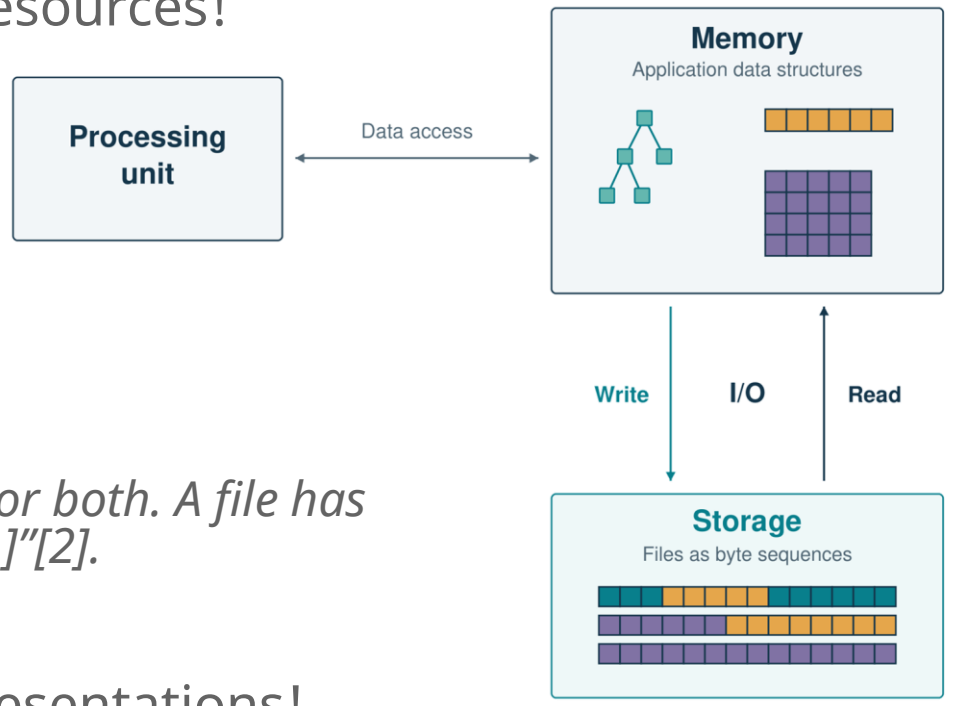


Acknowledgement: Parts of this presentation are adapted from the NHR I/O lecture by Sebastian Oeste, Center for Information Services and High Performance Computing (ZIH), TU Dresden, January 2022.

Introduction to I/O

INTRODUCTION TO I/O
What is I/O?

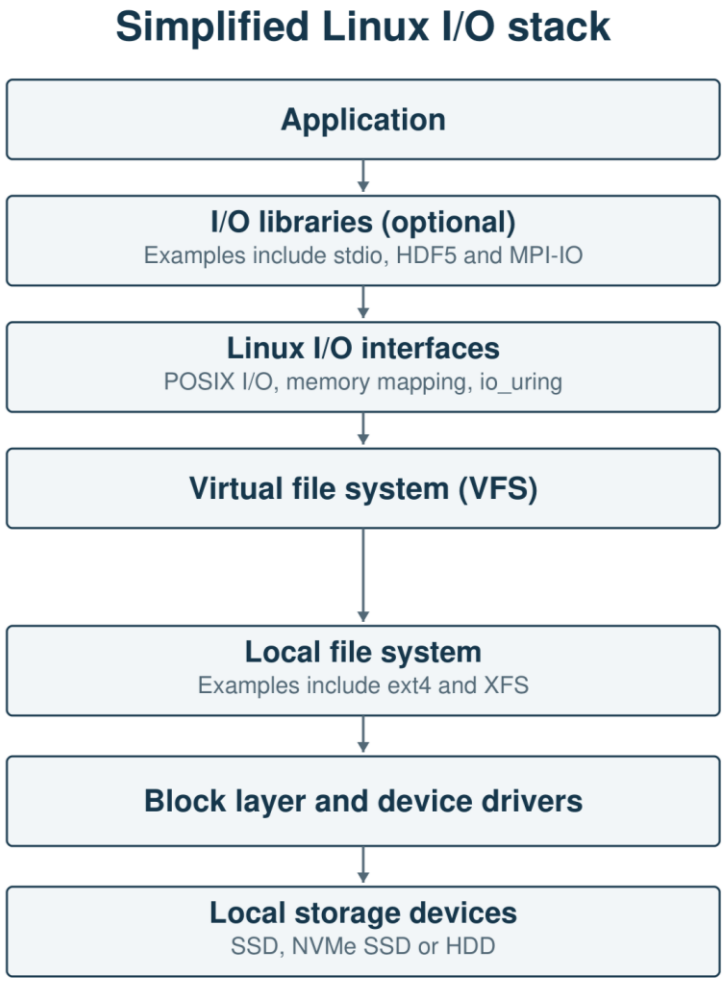
- I/O moves data between an application and external resources!
- Between data in memory and on disks
- In-Memory data has an application defined structure.
- On disks, data is typically stored in files.
- What is a File?
  - Unix philosophy says *„Everything is a file“*.
  - Linus Torvalds says *„Everything is a stream of bytes“[1].*
  - POSIX says *„An object that can be written to, or read from, or both. A file has certain attributes, including access permissions and type. […]”[2].*
- A regular file exposes a sequence of bytes.
- Applications and I/O libraries map between these representations!



[1] https://yarchive.net/comp/linux/everything_is_file.html
[2] IEEE Standard for Information Technology–Portable Operating System Interface (POSIX(R)) Base Specifications, Issue 8. IEEE Std 1003.1-2024 (Revision of IEEE Std 1003.1-2017) (Mar 2026).

INTRODUCTION TO I/O
I/O Stack

- Applications access files through interfaces such as POSIX I/O.
- Libraries such as libc provide wrappers around system calls.
- System calls enter the operating-system kernel.
- The virtual file system (VFS) provides a common interface to file-system implementations.
- Local file systems map file contents to storage blocks.
- The block layer and device drivers communicate with storage devices.



INTRODUCTION TO I/O
POSIX and POSIX I/O

- POSIX stands for *Portable Operating System Interface*.
- POSIX defines a standard way for an application program to obtain basic services from the operating system.
  - Tools, APIs but also semantic requirements, such as consistency.
  - Covers I/O, processes, signals and other operating-system services.
- Linux is mostly POSIX-compliant.
- POSIX-I/O defines an interface to work with files.
  - read(), write(), open(), close(), stat(), mkdir(), …
  - Defines also strong consistency requirements for data and metadata.
- POSIX was **not** designed with parallelism in mind.
  => *POSIX I/O does not describe collective access patterns across distributed processes.*

IEEE Standard for Information Technology–Portable Operating System Interface (POSIX(R)) Base Specifications, Issue 8. IEEE Std 1003.1-2024 (Revision of IEEE Std 1003.1-2017) (Mar 2026).

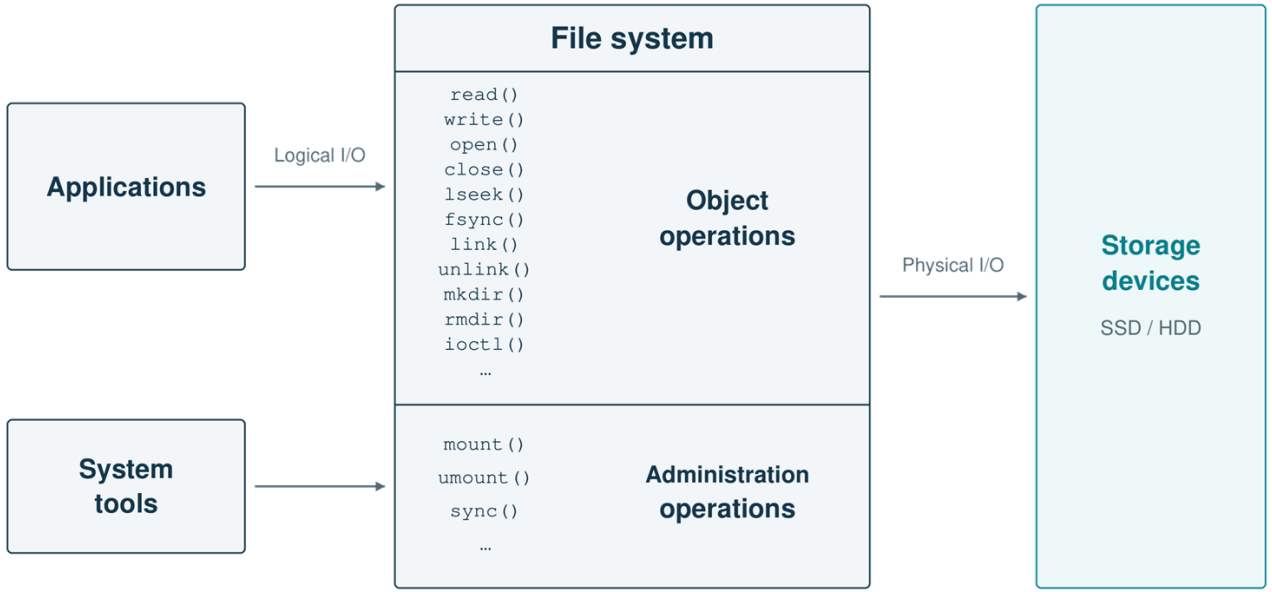
INTRODUCTION TO I/O
Files

- Files consist of data and metadata.
- Metadata include
  - Type of file
  - Permissions (rwxrwxrwx)
  - Timestamps
  - Size
  - Owner / Group
- Regular files support access at random byte offsets.
- Data is commonly stored in regular files.
- Files are organized by file systems.

Types of Files
Regular file (-)
Directory (d)
Block device (b)
Character device (c)
Symbolic link (l)
Socket (s)
FIFO (named pipe) (p)

INTRODUCTION TO I/O
Role of the File System

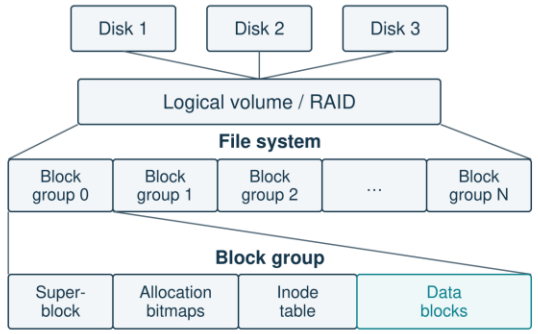
- A file system holds a collection of files.
- Organizes files within a namespace, usually a directory hierarchy.
- Stores file contents and manages metadata.
- Integrates into the namespace through mounting.
- Maps logical file accesses to underlying storage operations.

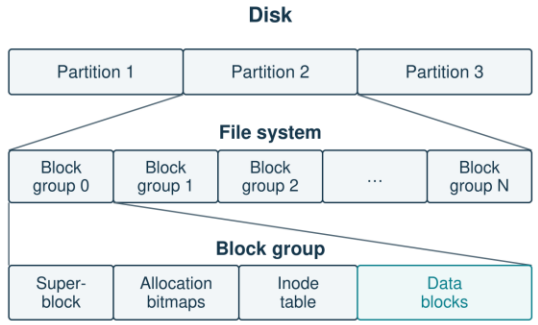


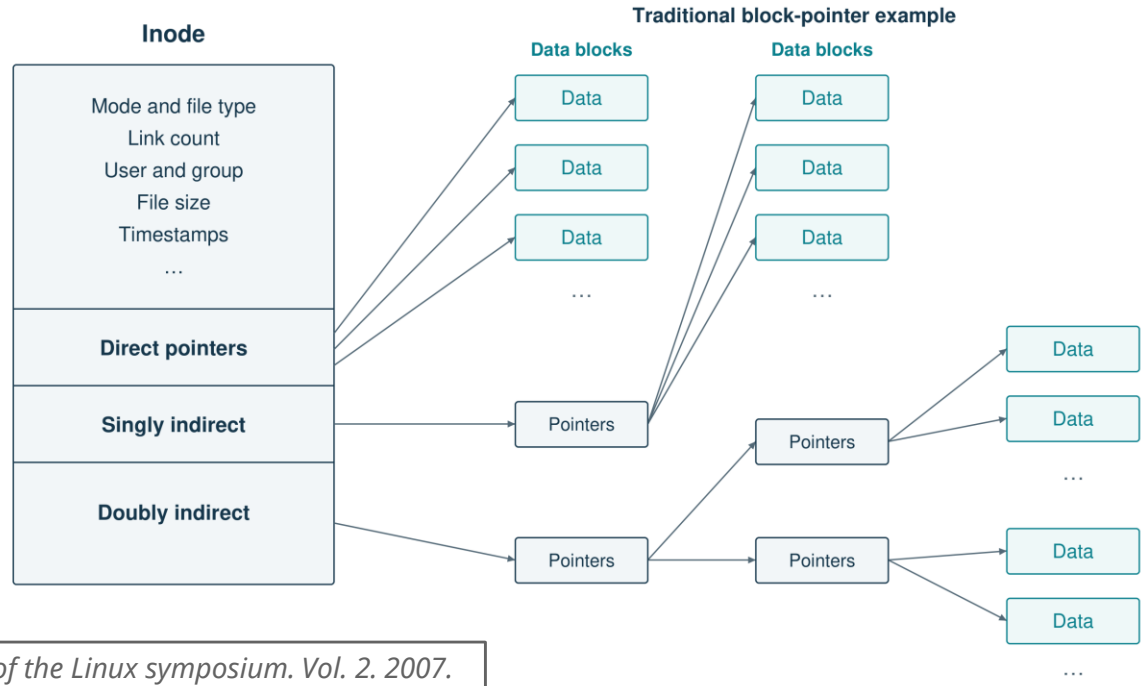
INTRODUCTION TO I/O
Inode Structure

- Disks can be partitioned with different file systems
- Disks can be grouped together to one file system
- Superblock stores general file system metadata
- File metadata is stored in inode structure
  - Record for metadata
  - Information to data blocks
  - Each inode within a file system has a unique number
- Files and directories consume inodes. Multiple hard links can refer to the same inode.
- No free space for inodes => ENOSPC
- How inodes are allocated depends on file system

- Hint: Check for free inodes with df -i



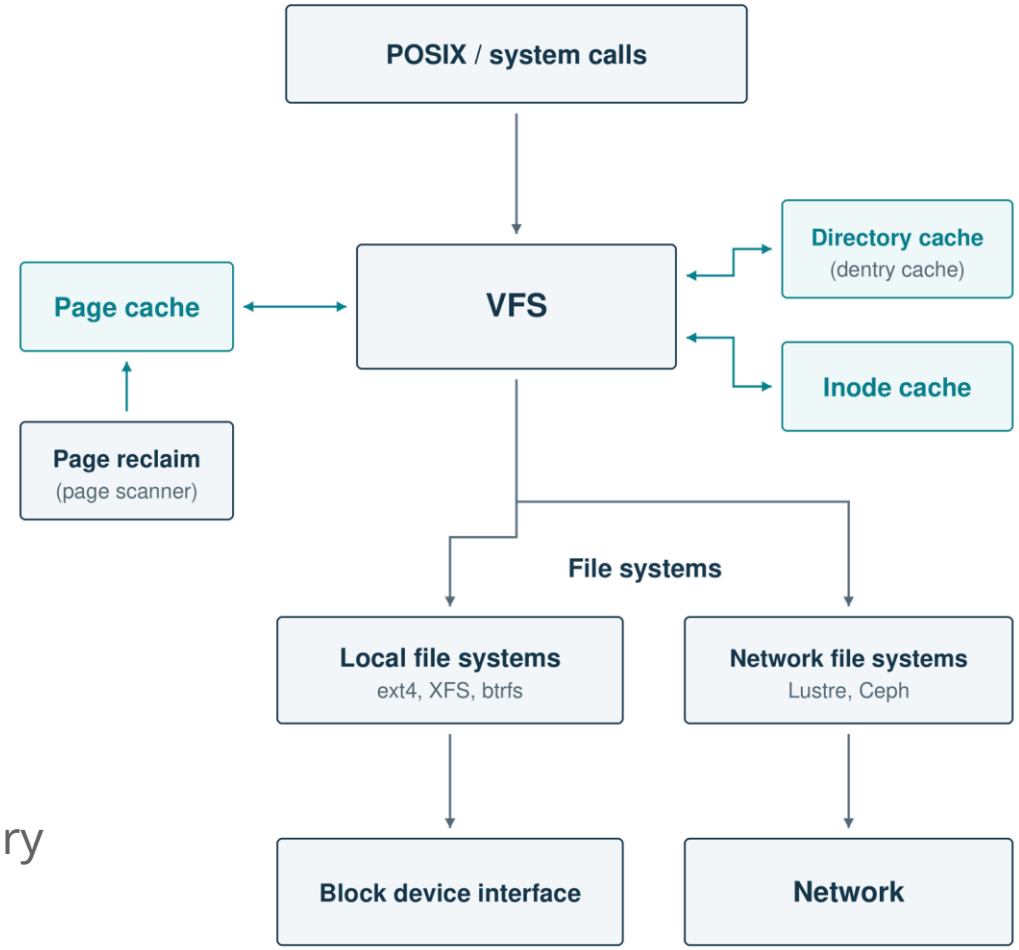




Mathur, Avantika, et al. "The new ext4 filesystem: current status and future plans." Proceedings of the Linux symposium. Vol. 2. 2007.

INTRODUCTION TO I/O
Linux File System Caches

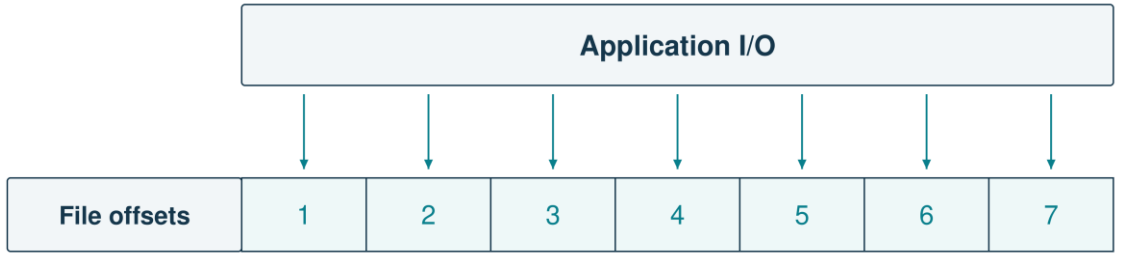
- Multiple caches improve local I/O performance
- Page Cache
  - Caches file data in memory
  - Modified pages are marked dirty
  - Writeback occurs in the background, under memory pressure, or through explicit synchronization such as fsync().
- Directory Cache (dentry cache)
  - Caches pathname-component lookups, including mappings to inodes
  - Improves performance for path lookups
  - Dynamically sized
- Inode Cache
  - Keeps inode objects and associated metadata in memory
  - Improves performance e.g. for stat() and open()
  - Most lookups will be done via dentry cache



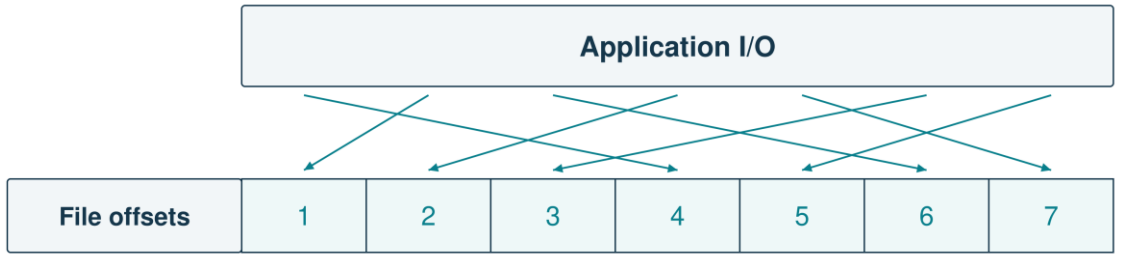
INTRODUCTION TO I/O
I/O Patterns

- Different I/O patterns depending on file offset
- File systems may use read-ahead to accelerate predictable reads
- ***Sequential I/O:*** accesses consecutive regions of a file
- ***Random I/O:*** accesses regions without a predictable sequential order

POSIX defines a File offset as:
"The byte position in the file where the next I/O operation begins. Each open file description associated with a regular file, block special file, or directory has a file offset."



Sequential I/O.

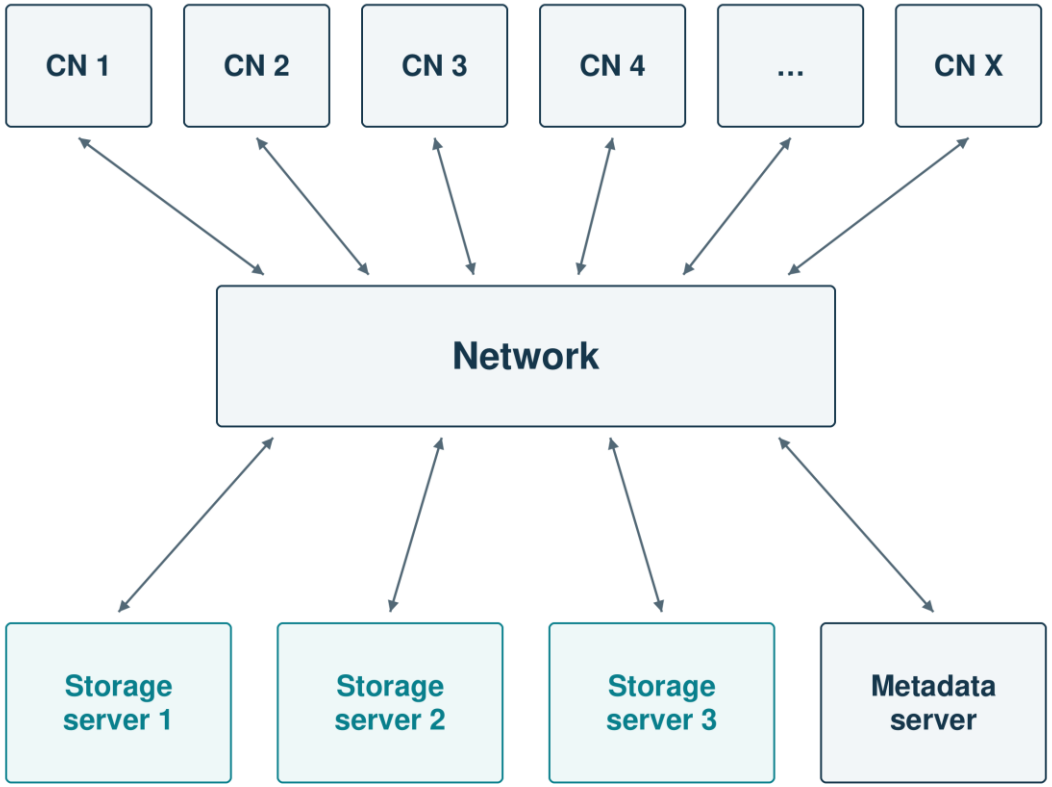


Random I/O.

Parallel File Systems

PARALLEL FILE SYSTEMS
Introduction

- Provide shared access to files from multiple compute nodes
- Present a common namespace to clients
- Distribute file data across storage targets to provide aggregate bandwidth => “striping”
- Support concurrent access by many processes
- Provide familiar file access through a mounted file system



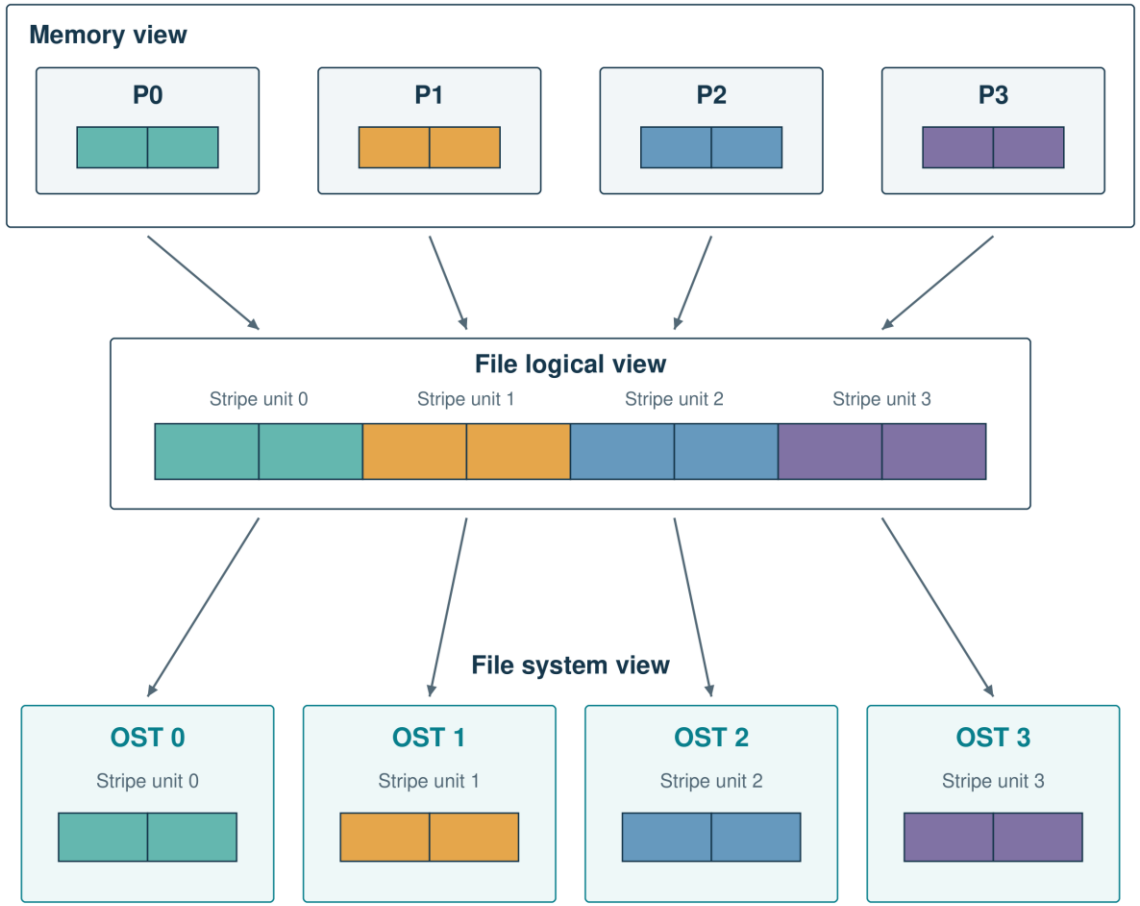
PARALLEL FILE SYSTEMS
Glossary

Abbreviation	Meaning	Description
MDS	Metadata Server	Handles namespace and metadata operations, such as file creation and directory lookup.
MDT	Metadata Target	Stores file-system metadata, including directory entries, attributes, and file layouts.
MGS	Management Server	Provides configuration information to (Lustre) clients and servers.
MGT	Management Target	Stores the configuration information managed by the MGS.
OSS	Object Storage Server	Handles file-data reads and writes for one or more OSTs.
OST	Object Storage Target	Stores data objects containing file contents.
Chunk	Data chunk / stripe unit	A consecutive portion of file data placed on one OST as part of striping.

PARALLEL FILE SYSTEMS
Striping

- A single file may be split into multiple chunks.
- Successive stripe-sized portions of a file are distributed across the selected OSTs.
- Advantages:
  - An increase in the bandwidth available when accessing the file.
  - An increase in the available disk space for storing the file.
- Disadvantages:
  - Increased overhead due to network operations and server contention.

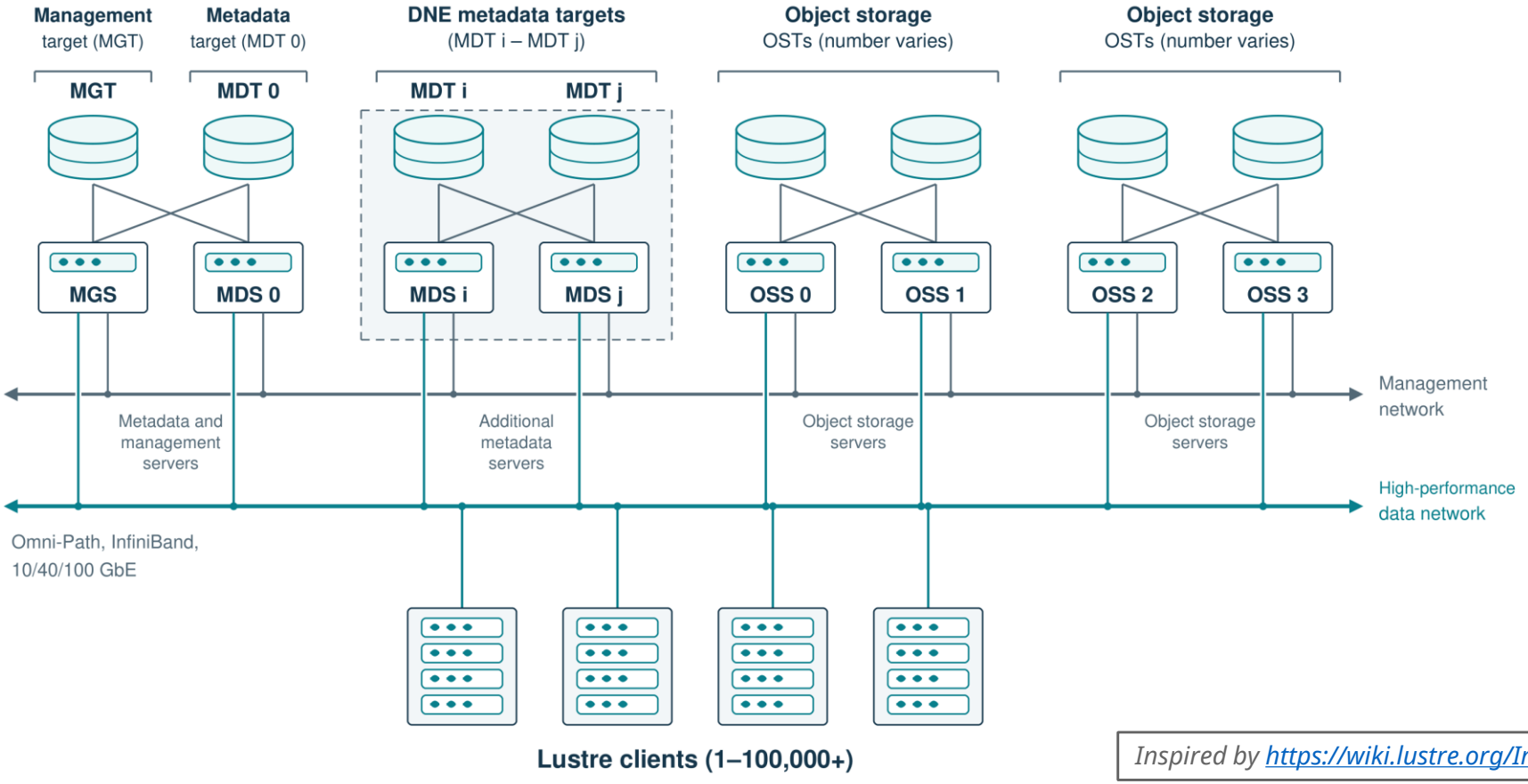
Users can often configure a file's stripe count and stripe size. Directory settings provide defaults for newly created files.



PARALLEL FILE SYSTEMS
Striping Considerations

- More OSTs can increase bandwidth but also add overhead.
- A single writer can benefit from striping until client, network or storage limits dominate.
- File-per-process workloads may achieve parallelism across files without wide striping of every file.
- Request size, alignment, stripe size and writer concurrency jointly affect performance.
- Excessive contention can reduce the benefit of additional OSTs.
- Start with site defaults and compare settings using a representative workload.

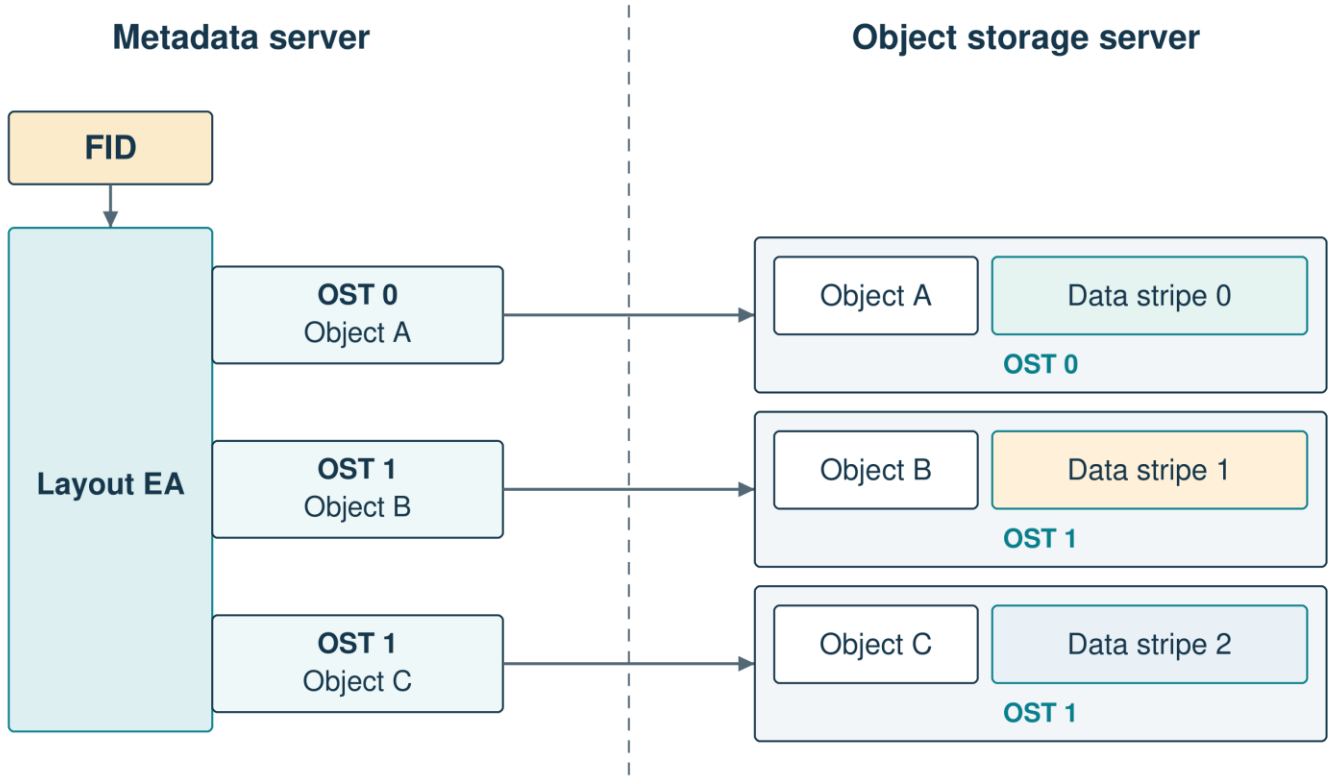
PARALLEL FILE SYSTEMS
Lustre



Inspired by https://wiki.lustre.org/Introduction_to_Lustre

PARALLEL FILE SYSTEMS
Lustre: Metadata

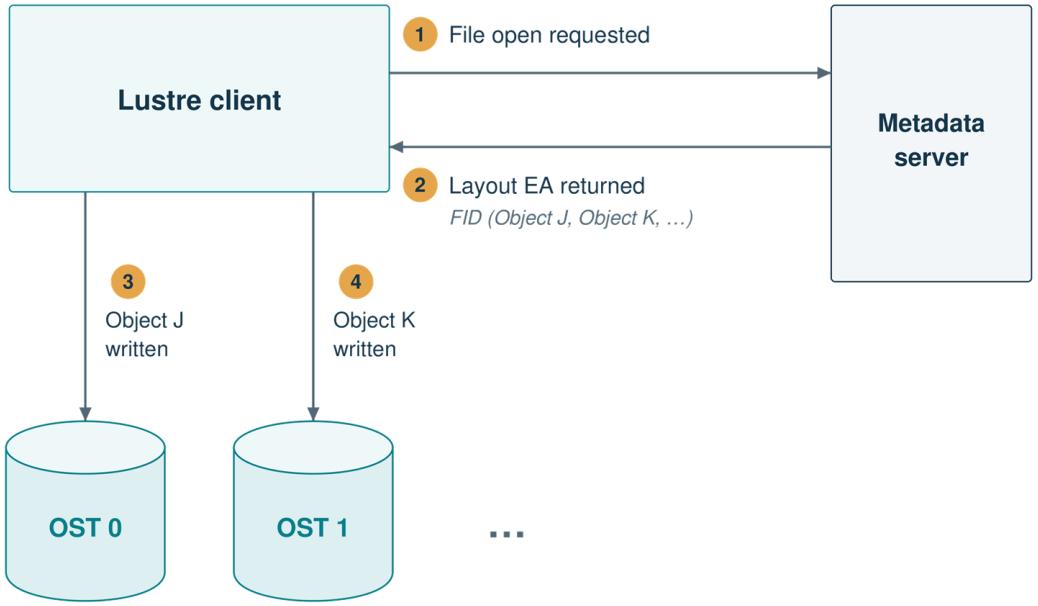
- The MDS manages namespace operations and metadata stored on MDTs
- Metadata includes ownership, permissions and file-layout information
- The layout identifies the storage objects and OSTs holding file data
- Lustre uses *File Identifiers* (FIDs) to identify files and objects



PARALLEL FILE SYSTEMS
Lustre: File I/O

- Clients must talk to both MDS and OSS servers
- Opening files, listing directories, … go to MDS
- File I/O goes directly to one or more OSSs

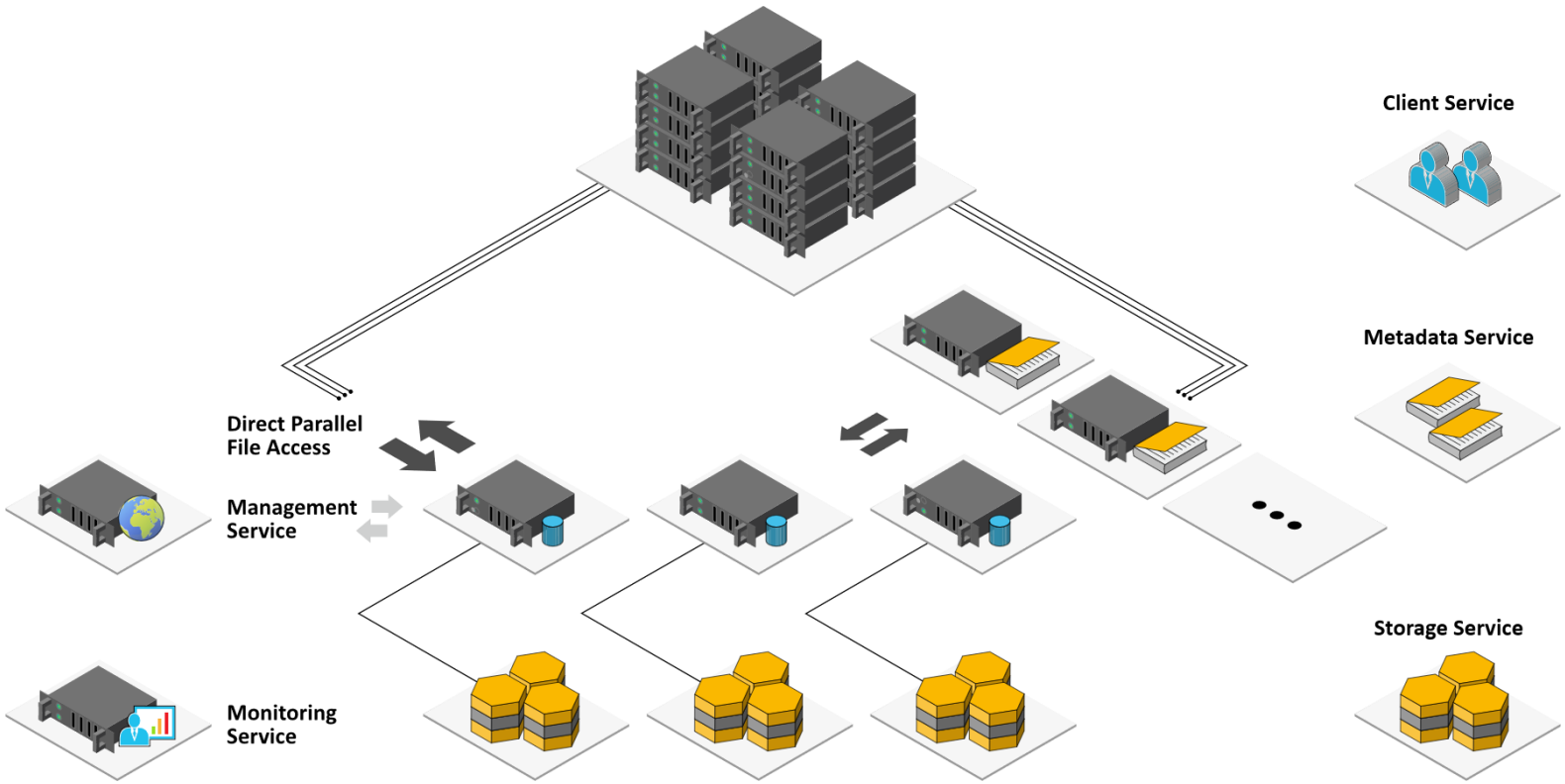
1. Client requests access to a file from the MDS.
2. The MDS returns file-layout and object information.
3. The client writes one portion of the data through the OSS serving OST0.
4. The client writes another portion through the OSS serving OST1.



PARALLEL FILE SYSTEMS
BeeGFS

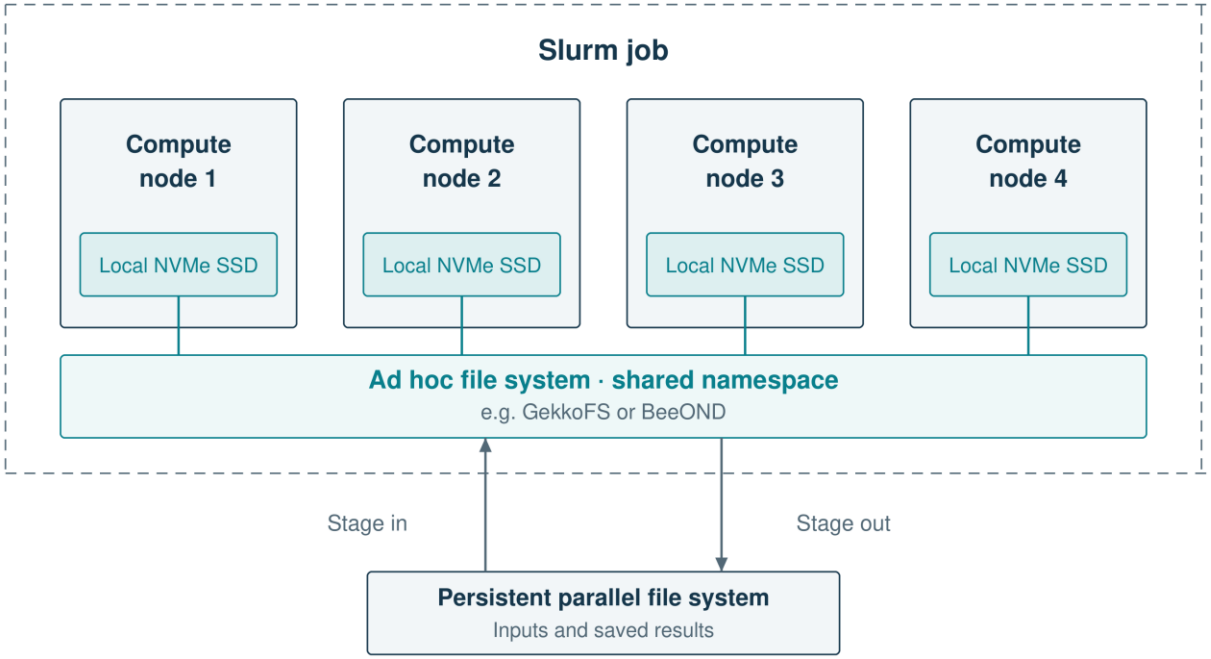
- Parallel file system with separate metadata and storage services
- Distributes file contents across multiple storage targets
- Allows clients to access storage servers concurrently
- Provides standard file access through a Linux kernel client
- Runs management, metadata and storage services in user space

https://doc.beegfs.io/latest/architecture/overview.html



PARALLEL FILE SYSTEMS
Ad-hoc File Systems for HPC

- Reduce contention on the shared parallel file system by offloading temporary and intensive I/O.
- Aggregate fast node-local storage, such as NVMe SSDs, into a shared namespace.
- Create storage instances for a job or workflow, integrated with the batch scheduler.
- Stage input data in and save required results before the temporary storage is removed.
- **Examples:** GekkoFS and BeeOND (BeeGFS On Demand).
- **Related approach:** DAOS can also support job-scoped ephemeral pools, alongside persistent deployments.

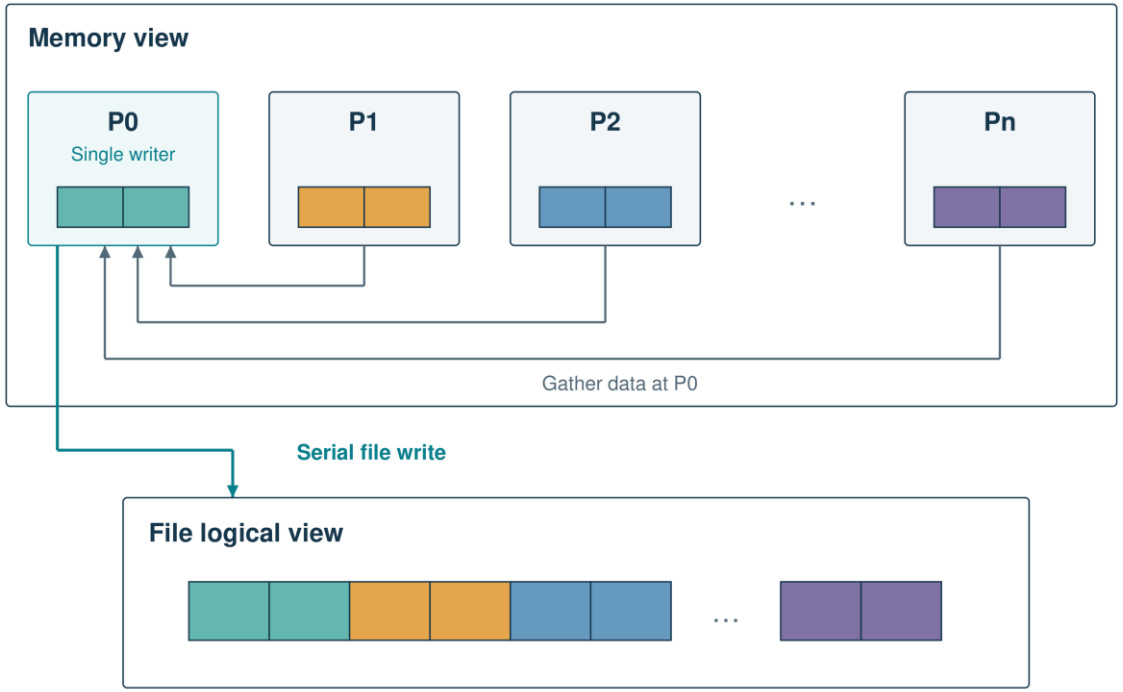


Brinkmann, André, Mohror, Kathryn, Yu, Weikuan, Carns, Philip, Cortes, Toni, Klasky, Scott A., Miranda, Alberto, Pfreundt, Franz-Josef, Ross, Robert B., and Vef, Marc-André. Ad Hoc File Systems for High-Performance Computing. United States: N. p., 2020. Web. https://doi.org/10.1007/s11390-020-9801-1

Introduction to Parallel I/O

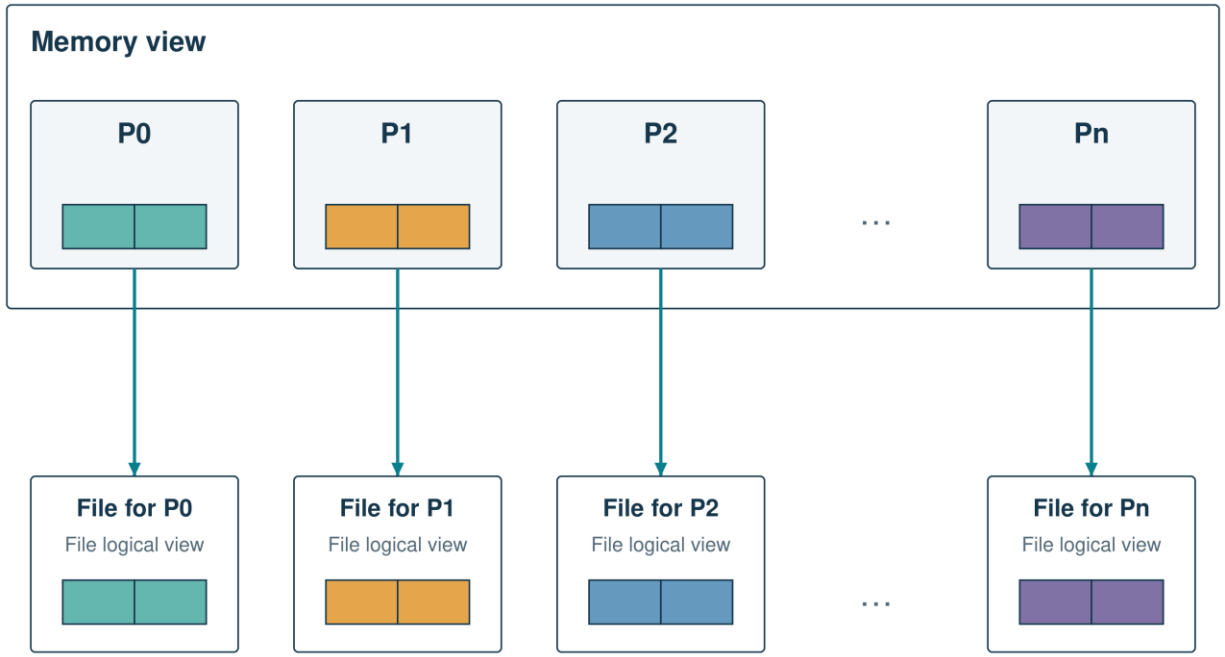
INTRODUCTION TO PARALLEL I/O
Single Writer (Serial) I/O

- Processes gather their data at one designated writer
- The writer stores the collected data in a single file
- Writer memory and communication costs can limit scalability
- All file I/O passes through one process
- Simple to implement, but often a bottleneck at scale



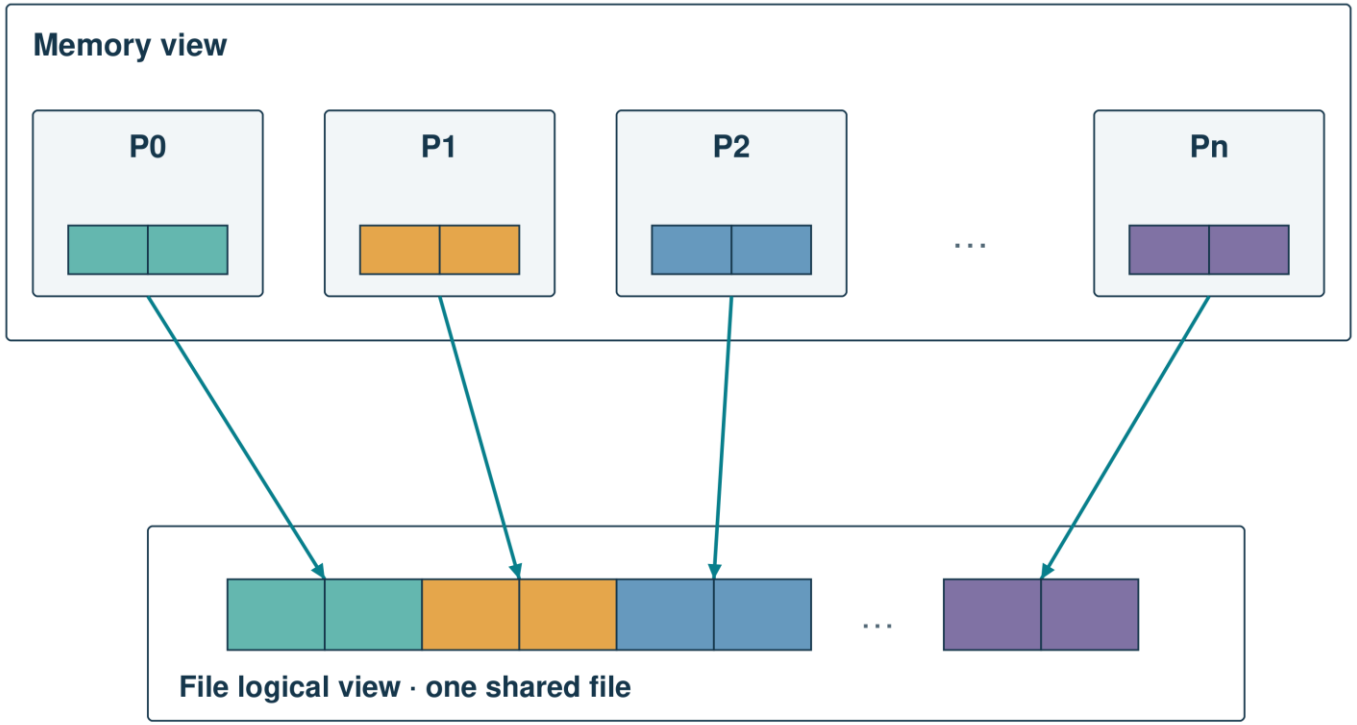
INTRODUCTION TO PARALLEL I/O
File-per-Process I/O

- Each process writes its own file
- A distributed dataset is stored across multiple files
- Analysis or restart may require combining or redistributing data
- Large file counts can increase metadata and file-management overhead
- Advantages:
  - Easy to implement
  - Can utilize parallel bandwidth
- Disadvantages:
  - Number of files can create bottleneck with metadata operations
  - Number of simultaneous disk accesses can create contention for file system resources



INTRODUCTION TO PARALLEL I/O
Shared-File I/O: Single File, Multiple Writers

- Each process independently accesses its assigned regions of one shared file
- Data layout within the shared file is important
- At large process counts contention can build for file system resources
- Correct offsets and coordination are essential when accesses overlap

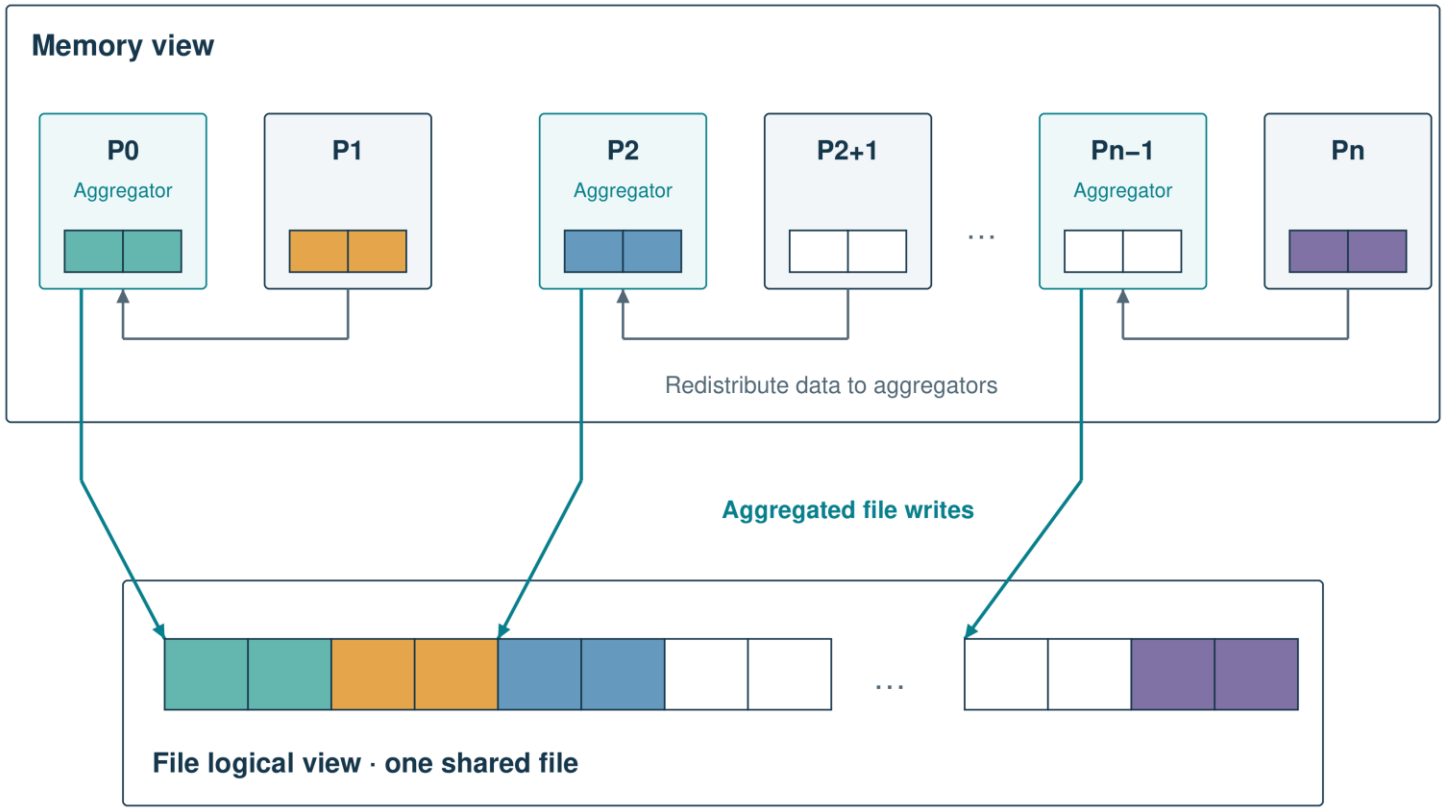


INTRODUCTION TO PARALLEL I/O
Shared-File I/O: Single File, Collective Writers

- Processes participate collectively in the I/O operation
- The library may redistribute data to selected aggregator processes
- Aggregators issue larger, coordinated accesses to the shared file
- Aggregation adds communication and buffering costs

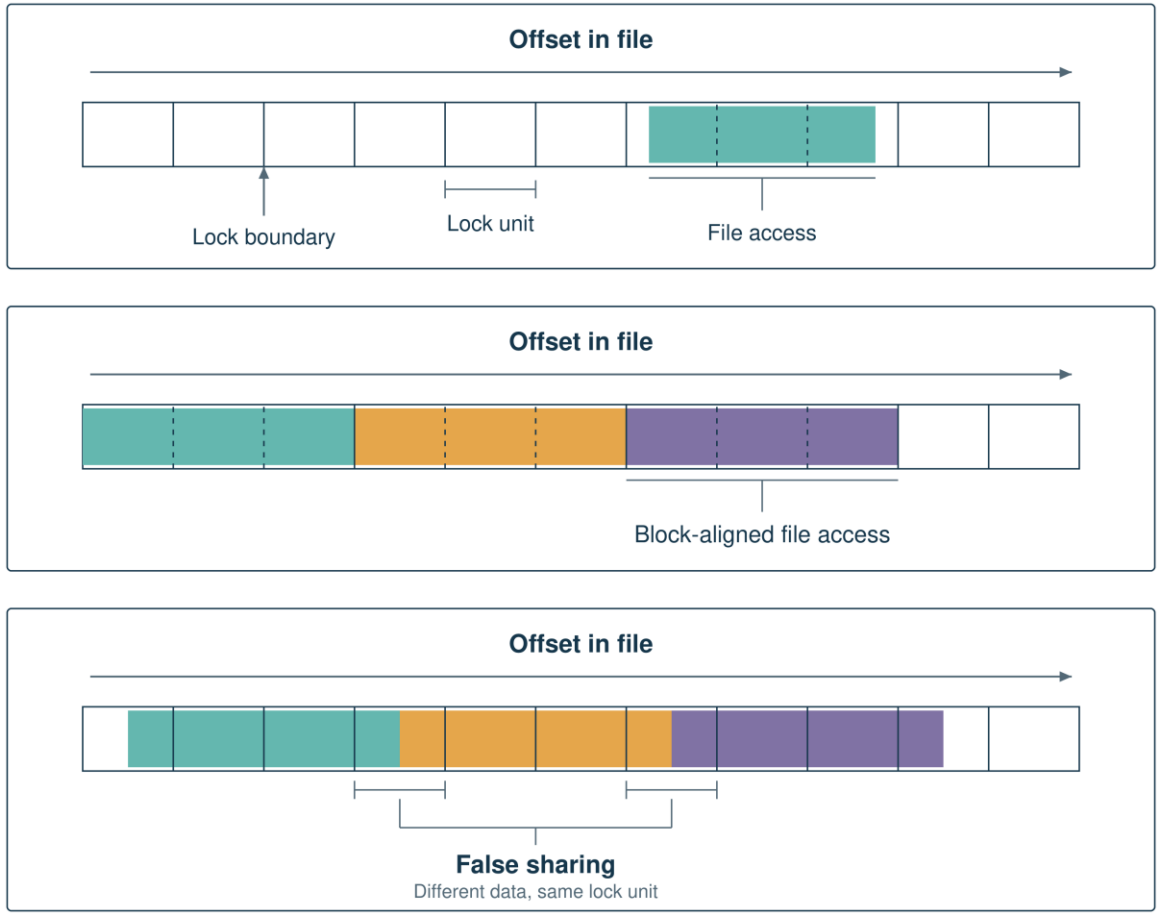
How to choose the right number of I/O processes?

The best aggregator count depends on the access pattern, network, storage layout and MPI implementation.



INTRODUCTION TO PARALLEL I/O
Managing Concurrent Access: File-System Locking

- Parallel file systems such as Lustre use internal locks to coordinate access and caching
- Lock coverage depends on the implementation and access pattern
- Disjoint application accesses can still contend when they share a lock region
- Conflicting access can require lock revocation and cache writeback or invalidation
- Coordinated, aligned accesses can reduce lock contention



INTRODUCTION TO PARALLEL I/O
MPI-IO

- Provides a low-level interface to carrying out parallel I/O
- Facilitate concurrent access by groups of processes

MPI-IO can be done in two basic ways:

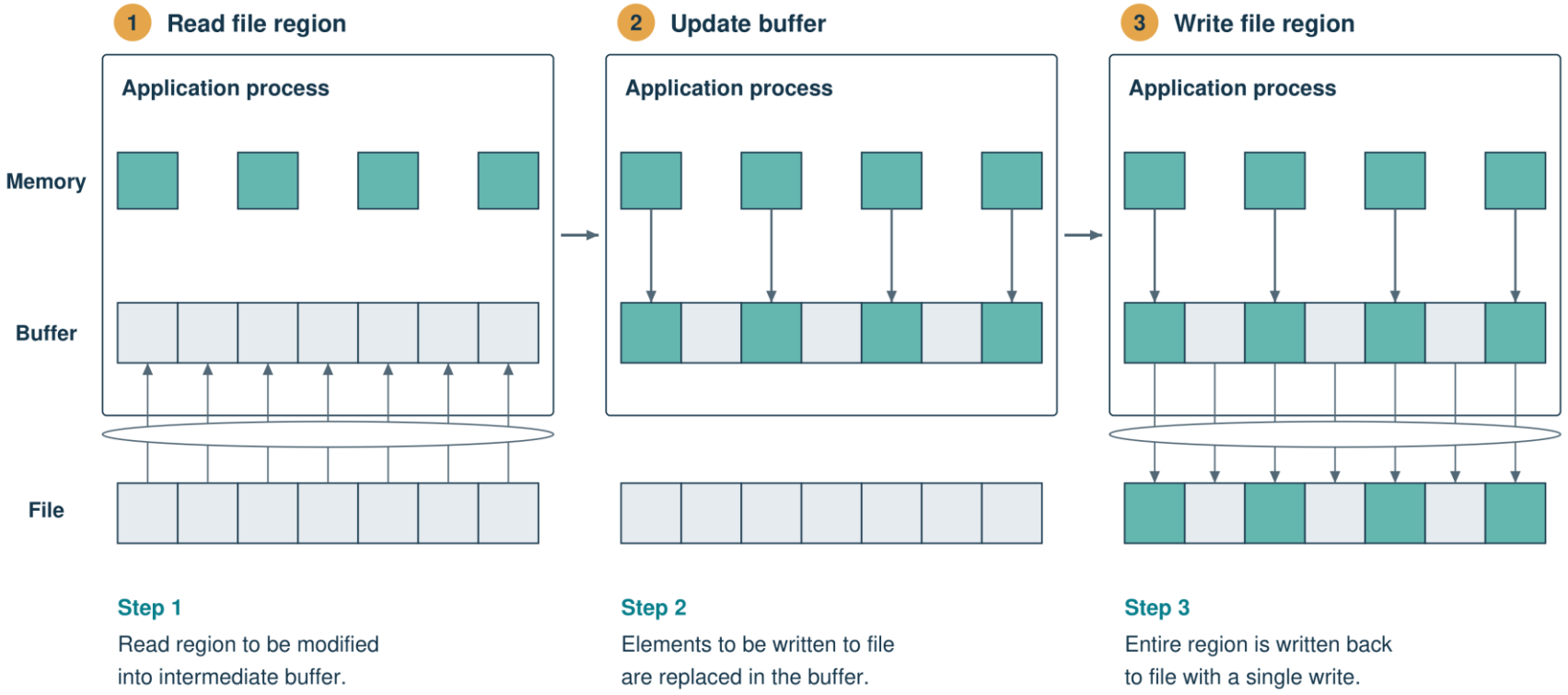
- **Independent MPI-I/O:**
  - Each process issues noncollective data-access calls, such as MPI_File_write() or MPI_File_read()
  - Like POSIX-I/O, but supports derived datatypes and thus noncontiguous data and nonuniform strides and can take advantage of MPI_Info hints
- **Collective MPI-I/O:**
  - All processes associated with the file handle must call matching collective operations, including processes with zero data to transfer. E.g. MPI_File_write_all()/MPI_File_read_all()
  - This allows the MPI library to do I/O optimization

INTRODUCTION TO PARALLEL I/O
Collective I/O with MPI-IO

- All processes in the file's communicator participate in each collective operation.
- Each process describes its own data and file regions.
- MPI-I/O library is given a lot of information.
  - Collection of processes reading or writing data.
  - Structured description of the regions.
- When writing in collective mode, the MPI library carries out a number of optimizations
  - Using fewer processes to actually do the writing – typically one per node.
  - It aggregates data in appropriate chunks before writing.
- MPI_Info hints can influence implementation-specific tuning.

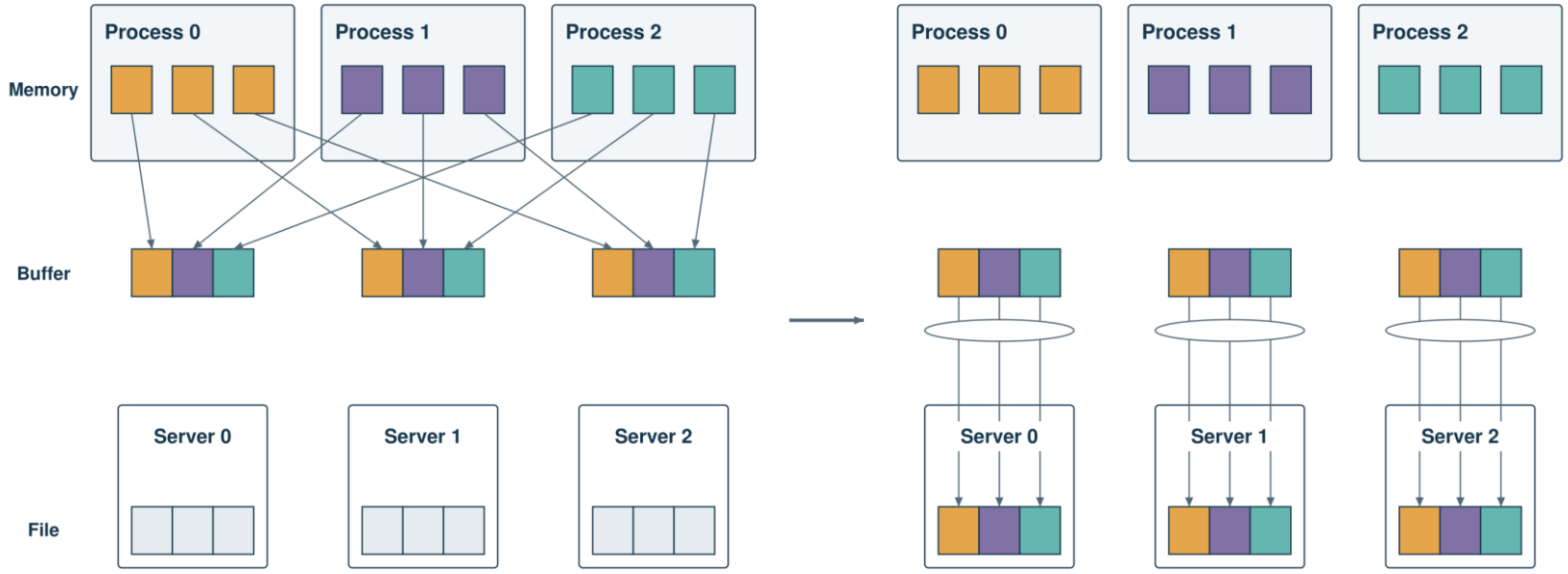
INTRODUCTION TO PARALLEL I/O
Data Sieving

- Combines noncontiguous accesses into fewer, larger file operations.
- Transfers intervening bytes through an intermediate buffer.
- For writes, read-modify-write preserves data in the gaps.
- Extra data movement and locking can outweigh the benefit.



INTRODUCTION TO PARALLEL I/O
Two-Phase I/O

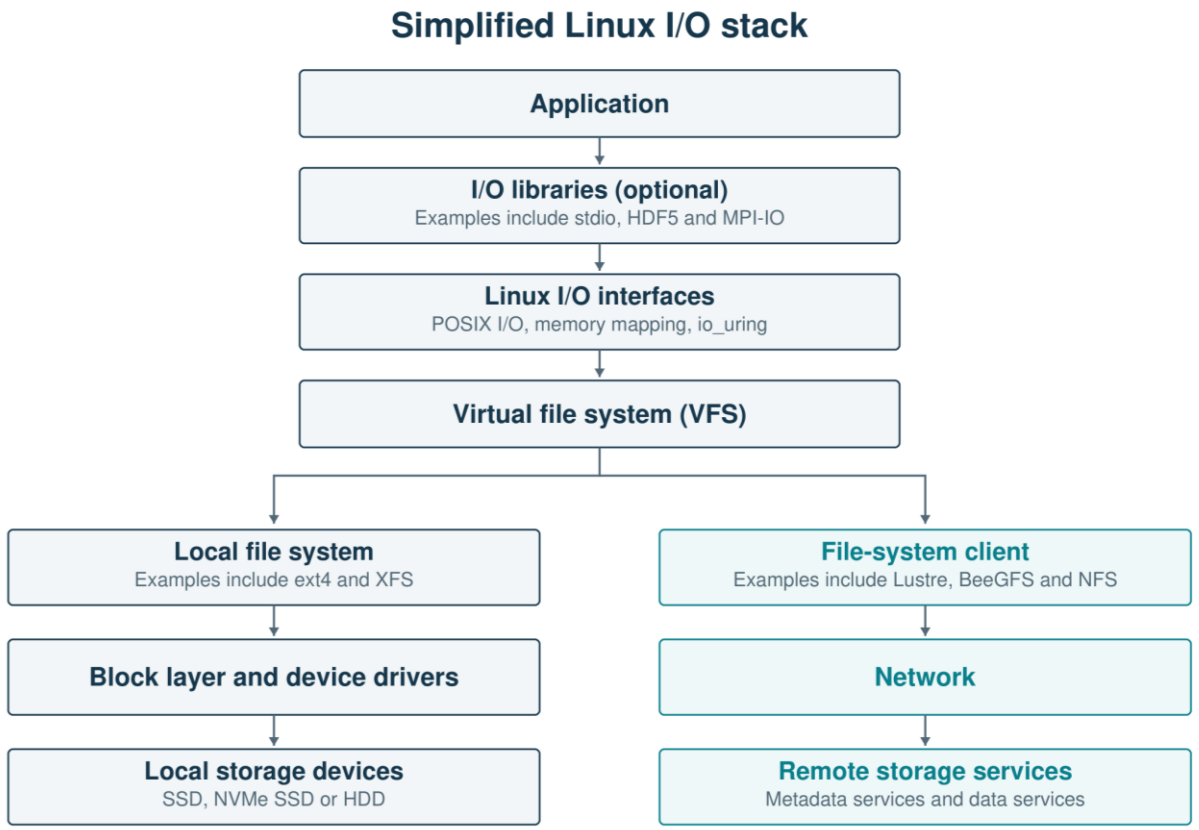
- Redistributes data among processes according to file regions.
- Aggregators issue larger, more contiguous file accesses.
- Adds communication and buffering costs that must be balanced against I/O savings.

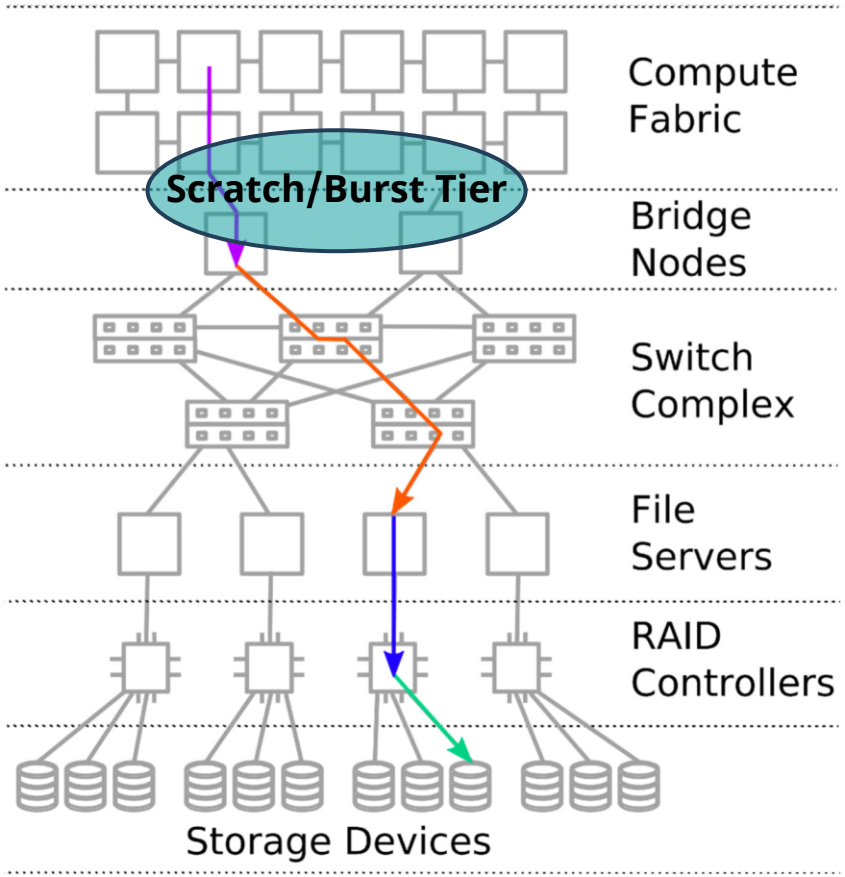


Liao, Wei-keng, and Alok Choudhary. "Dynamically adapting file domain partitioning methods for collective I/O based on underlying parallel file system locking protocols." SC'08: Proceedings of the 2008 ACM/IEEE Conference on Supercomputing. IEEE, 2008.

Understanding Parallel I/O Performance

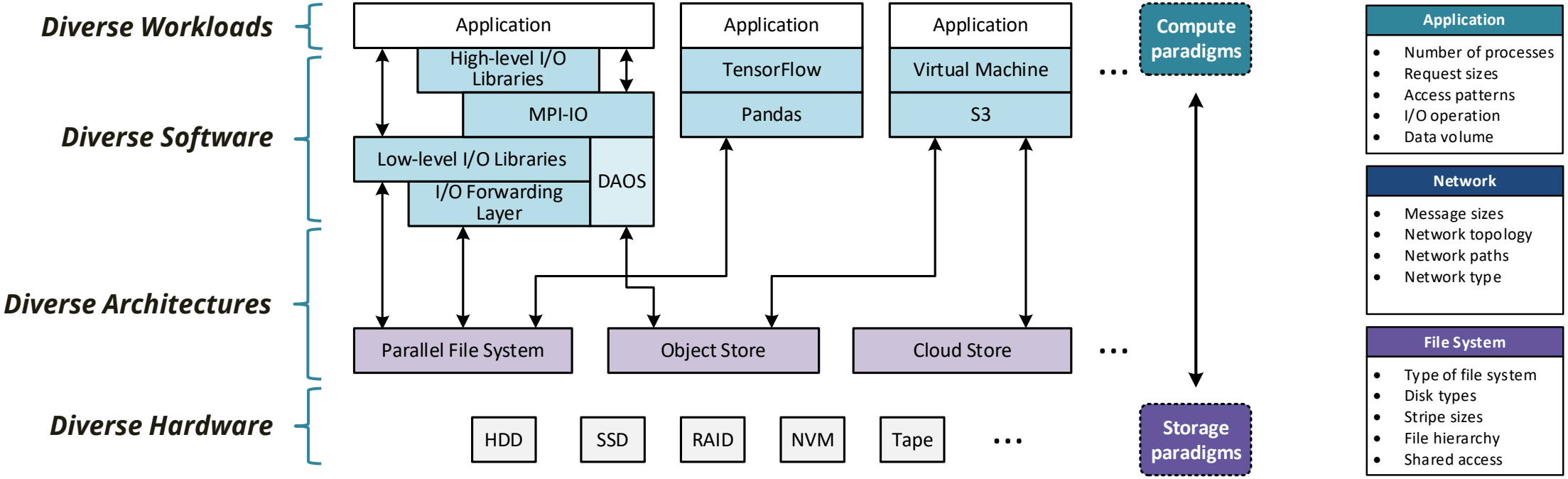
UNDERSTANDING PARALLEL I/O PERFORMANCE
I/O Stack versus Hardware Architecture





UNDERSTANDING PARALLEL I/O PERFORMANCE
Why the Problem is Getting Harder

HPC + AI workflows increasingly data-intensive



I/O bottlenecks limit scalability and throughput

Profiling & tracing tools essential for optimization

I/O Best Practices

I/O BEST PRACTICES
Less I/O and Metadata Overhead

- Write only the data needed for analysis, restart and reproducibility
- Choose output and checkpoint frequency deliberately
- Avoid repeated file creation, opening, closing and directory scans
- Read small, shared input files once and distribute their contents where appropriate
- Limit excessive per-process logging and large collections of tiny files

4,000,000 files

4,000 ranks × 10 files × 100 output steps

Illustrative workload, assuming new files at every output step

I/O BEST PRACTICES
Access Patterns and Parallel Writers

- Transfer arrays or blocks instead of individual elements
- Combine small, scattered requests into larger, contiguous operations
- Choose serial, file-per-process or shared-file I/O according to workload and scale
- Use collective I/O when aggregation improves the access pattern
- Coordinate shared-file writes to avoid overlapping accesses and false sharing

262,144 vs 256 requests

1 GiB in 4 KiB requests vs 4 MiB requests

Illustrative request counts, not a predicted speedup

I/O BEST PRACTICES
Data Layout and Storage Configuration

- Use the appropriate storage tier for temporary data, checkpoints and retained results
- Start with the center's recommended file-system settings
- Tune Lustre stripe count and stripe size for the actual access pattern and concurrency
- With HDF5 or similar libraries, match dataset layout and chunking to how data are accessed
- When using temporary job storage, plan staging and save required results before teardown

1, 4, 8 stripes (i.e., OSTs per file)

Example test set for one representative workload

Follow site guidance and available resources

I/O BEST PRACTICES
Measurement and Verification

- Establish a baseline with a representative workload
- Examine I/O time, request sizes, file counts and differences between processes
- Change one factor at a time and repeat measurements
- Include communication, staging and required synchronization in the comparison
- Check correctness and restartability as well as performance

2 GiB/s vs 1 GiB/s

10 GiB / 5 s transfer vs 10 GiB / 10 s full I/O phase

Hypothetical timings with different measurement boundaries

Thank you!

Contact: neuwirth@uni-mainz.de

Website: https://nhrsw.de/

NHR SÜD-WEST

JOHANNES GUTENBERG UNIVERSITÄT MAINZ

JG|U